

Chapter 3 – Customizing Map Reduce Execution and Implementing Map Reduce Program

Topic covered

Controlling MapReduce Execution with InputFormat, Reading Data with Custom Record Reader, Organizing Output Data with OutputFormats

- Customizing Data with RecordWriter, Optimizing MapReduce Execution with Combiner, Controlling Reducer Execution with Partitioner
- Customizing the MapReduce Execution in Terms of YARN Implementing a MapReduce Program for Sorting Text Data.

Hadoop Input Format, Types of Input Format in Map Reduce

In Hadoop MapReduce, the InputFormat is responsible for reading data from the input sources and providing it to the Mapper for processing. Hadoop provides several built-in InputFormats to handle different types of input data. Here are some common types of InputFormats in Hadoop MapReduce:

1. **TextInputFormat:**

- This is the default InputFormat in Hadoop.
- It treats each line of the input file as a record and provides the Mapper with a key-value pair where the key is the byte offset of the line within the file and the value is the content of the line.

2. **KeyValueTextInputFormat:**

- This InputFormat is similar to TextInputFormat, but it expects each line of the input file to be in the form of "key<delimiter>value".
- It parses each line into key-value pairs based on the specified delimiter.

3. **SequenceFileInputFormat:**

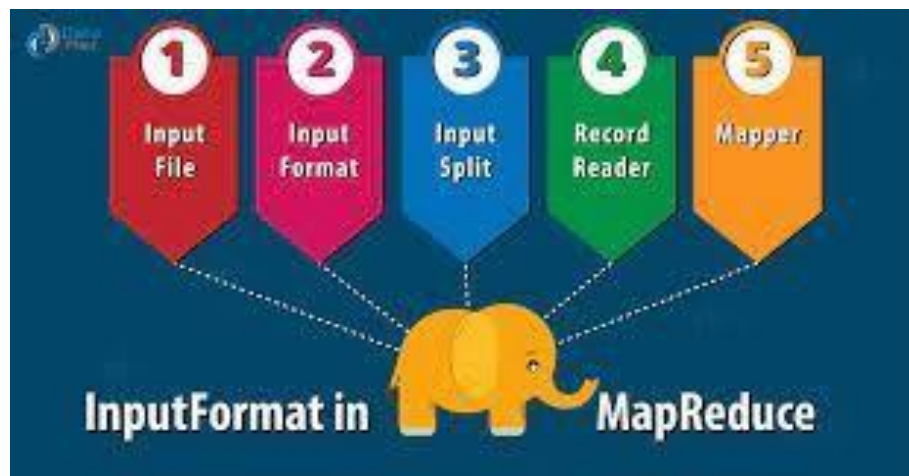
- This InputFormat is used for reading data stored in Hadoop's binary file format called SequenceFile.



- SequenceFile is a flat file containing binary key-value pairs.
 - SequenceFileInputFormat provides the Mapper with key-value pairs extracted from the SequenceFile.
4. **TextInputFormatWithLineNum:**
- This is a custom InputFormat provided by Hadoop.
 - It extends TextInputFormat and includes the line number as a key along with the content of each line as the value.
5. **NLineInputFormat:**
- This InputFormat is used when you want to control the number of lines processed by each Mapper.
 - It splits the input data into fixed-size chunks of N lines and assigns each chunk to a separate Mapper.
6. **CombineFileInputFormat:**
- This InputFormat is used for processing multiple small files efficiently.
 - It combines multiple small files into larger input splits to reduce the overhead of spawning individual map tasks for each small file.

These InputFormats cater to different data sources and formats, allowing MapReduce jobs to process a wide range of input data efficiently. The choice of InputFormat depends on the nature of the input data and the requirements of the MapReduce job.

Hadoop Input Format, Types of Input Format in Map Reduce



What is Hadoop Input Format?

In Hadoop, an `InputFormat` is an interface responsible for defining how input data is read and split into input splits, which are then assigned to individual map tasks for processing in a MapReduce job. It specifies the input source, such as a file or database, and how that source is divided into manageable chunks for processing by multiple mappers in a distributed environment.

The `InputFormat` interface defines two main methods:

1. **`getSplits()`**: This method is responsible for dividing the input data into splits, each of which corresponds to a portion of the input data that will be processed by a single mapper. The `getSplits()` method typically returns a list of `InputSplit` objects, where each `InputSplit` represents a portion of the input data.
2. **`createRecordReader()`**: This method is responsible for creating a `RecordReader` object, which is used by the mapper to read and process the records within each input split. The `createRecordReader()` method initializes the `RecordReader` with the input split and other necessary parameters.

Hadoop provides several built-in implementations of the `InputFormat` interface to handle different types of input data, such as text files, binary files, and databases. These built-in `InputFormats` include `TextInputFormat`, `KeyValueTextInputFormat`, `SequenceFileInputFormat`, and others. Additionally, users can define custom `InputFormats` to handle specialized data formats or sources that are not supported by the built-in `InputFormats`.

Overall, the `InputFormat` in Hadoop plays a crucial role in managing input data and facilitating the parallel execution of MapReduce jobs across a distributed cluster. It abstracts away the details of input data management and provides a consistent interface for reading and processing data in a distributed environment.



Types of Input Format in Map Reduce



Customizing Data with Record Writer, Optimizing Map Reduce Execution with Combiner, Controlling Reducer Execution with Partitioner

Customizing Data with Record Writer: The RecordWriter in Hadoop MapReduce is responsible for writing the output of the Mapper tasks to the appropriate output location, typically HDFS (Hadoop Distributed File System). It serializes the key-value pairs produced by the Mapper and writes them to the output file or files. Customizing the RecordWriter allows developers to control how the output data is formatted and written.

1. **OutputFormat:** The RecordWriter is typically used in conjunction with an OutputFormat, which defines how the output data is organized and written. Hadoop provides several built-in OutputFormats, such as TextOutputFormat for plain text files, SequenceFileOutputFormat for binary SequenceFiles, and others. Custom OutputFormats can also be implemented to handle specialized output requirements.
2. **Custom RecordWriter:** Developers can implement custom RecordWriters to customize how the output data is written. This may involve defining custom serialization formats, applying compression techniques, or integrating with external storage systems. Custom RecordWriters are often

used to optimize performance, reduce storage costs, or meet specific output formatting requirements.

What is the sequence of execution of the Mapper, Combiner, and Partitioner?

The sequence of execution of Mapper, Combiner, and Partitioner in Hadoop MapReduce is as follows:

1. Mapper:

- The Mapper task is the first stage of the MapReduce job execution.
- Input data is split into input splits, and each split is processed by a separate Mapper task.
- The Mapper task reads input data, applies the user-defined map function to each input record, and emits intermediate key-value pairs.

2. Combiner (optional):

- The Combiner task is an optional optimization that occurs after the Mapper stage and before the Shuffle and Sort phase.
- The output of each Mapper task is passed through the Combiner function before being shuffled and sent to the Reducers.
- The Combiner performs local aggregation of intermediate key-value pairs with the same key.
- It reduces the volume of data transferred over the network to the Reducers by aggregating data locally within each Mapper task.

3. Partitioner:

- The Partitioner determines which Reducer instance will receive each intermediate key-value pair produced by the Mapper.
- It operates after the Map and Combine stages and before the Shuffle and Sort phase.
- The Partitioner uses a hash-based or custom partitioning logic to route keys to Reducers based on their hash value or other partitioning criteria.
- The goal of the Partitioner is to evenly distribute intermediate data across Reducer tasks to facilitate parallel processing and load balancing.

4. Shuffle and Sort:

- After the Mapper, Combiner (if used), and Partitioner stages, the Shuffle and Sort phase begins.
- In this phase, the intermediate key-value pairs produced by the Mapper tasks are shuffled and sorted based on their keys.
- Data with the same key is grouped together and sorted before being sent to the appropriate Reducer tasks.
- The Shuffle and Sort phase ensures that all values associated with a particular key are processed together by the Reducer tasks.

5. Reducer:

- The Reducer task is the final stage of the MapReduce job execution.
- Each Reducer task receives a subset of the intermediate key-value pairs produced by the Mapper tasks, as determined by the Partitioner.
- The Reducer task applies the user-defined reduce function to each group of values with the same key, producing the final output of the MapReduce job.

Overall, the sequence of execution involves the Mapper processing input data, optionally followed by the Combiner performing local aggregation, the Partitioner determining Reducer assignments, the Shuffle and Sort phase organizing data for Reducers, and finally, the Reducer producing the final output.

How do map, reduce and filter functions work in Python?

The 'Map' operation in programming refers to the transformation of data from one form into another. The 'Reduce' operation is a method for combining the elements of a list into a single result, while 'Filter' is a method for selecting a subset of items from a list based on a particular condition.

In summary, 'Map', 'Reduce', and 'Filter' operations in Computer Science refer to:

Map: Transformation of one form of data into another

Reduce: Combination of list elements into a single result

Filter: Selection of list subset based on a certain condition

The Fundamentals of Map Reduce and Filter

The map takes a function and an array as its inputs. It applies the function to each element in the array and returns an array of the results. The function also takes a function and an array as inputs, but it returns a single output value which is the result of the function applied sequentially to the elements of the array.

1. 'map' Function:

- The map function applies a specified function to each item in an iterable (e.g., a list) and returns an iterator that yields the results.
- Syntax: `map(function, iterable)`.
- Example:

Doubling each element in a list

```
numbers = [1, 2, 3, 4, 5]
```

```
doubled_numbers = list(map(lambda x: x * 2, numbers))print(doubled_numbers) #
```

Output: [2, 4, 6, 8, 10]

2. 'reduce' Function:

- The reduce function applies a rolling computation to pairs of values in an iterable, progressively reducing them to a single value.
- It was part of the built-in functions in Python 2 but is available in the `functools` module in Python 3.
- Syntax: `functools.reduce(function, iterable[, initializer])`.
- Example:

```
from functools import reduce
```

```
# Summing all elements in a list
numbers = [1, 2, 3, 4, 5]
total = reduce(lambda x, y: x + y, numbers)
print(total) # Output: 15
```

3. 'filter' Function

- The filter function constructs an iterator from elements of an iterable for which a specified function returns True.
- Syntax: filter(function, iterable).
- Example:

```
# Filtering even numbers from a list
numbers = [1, 2, 3, 4, 5]
even_numbers = list(filter(lambda x: x % 2 == 0, numbers))
print(even_numbers) # Output: [2, 4]
```

In summary:

- map transforms each item in an iterable using a specified function.
- reduce applies a rolling computation to pairs of values in an iterable, resulting in a single output.
- filter selects items from an iterable that satisfy a given condition, specified by a function.

These functions are particularly useful when you want to apply a transformation, aggregation, or filtering operation to a collection of data without writing explicit loops.

Difference between Map, Filter, and Reduce

Map, Filter, and Reduce are different in many ways though they are interlinked within the context of data manipulation in functional programming. Each has a specific task which it is designed to handle efficiently.

Contrasting features of map filter and reduce

At their very core, Map, Filter, and Reduce focus on transforming, selecting, and accumulating respectively.

Description	Purpose	Input	Output
The Map function, from a wide perspective, is about transformation. It applies a given function to each item of an array, creating a new array with the results.	Array transformation	An array and a function	A new array resulting from the application of the function on each element of the original array
The Filter function aims at selection. It constructs a new array that includes elements of the original array that meet a specified condition.	Array element selection	An array and a selection criterion function	A new array consisting only of elements from the original array that satisfy the condition
The Reduce function, as its name suggests, focuses on reduction. It processes an array to combine its values and reduce them down to a single value.	Array reduction into a single value	An array and a binary function	A single value is obtained by recursively applying the function on pairs of elements until only one value remains.

Guidelines on How to Use Map, Filter and Reduce Effectively

In order to reap the full benefits of Map, Filter, and Reduce, it's vital to understand their efficient usage and nuances. Here are some key guidelines: 1. Understand the

Concept: Map, Filter, and Reduce each serve specific purposes. Map is for transformation, Filter is for selection, and Reduce is for accumulation. Always consider what exactly is to be achieved with your data before employing one of these functions. 2. Combine Functions when Necessary: To optimize performance or readability, you can sometimes use these functions in conjunction. For instance, you may use Filter to select relevant items before applying Map or Reduce. 3. Employ Laziness: Lazy evaluation is a key aspect that can greatly improve performance. It involves evaluating expressions only as needed, which can be particularly beneficial when dealing with large data sets. 4. Readability over Brevity: Keep your code accessible and maintainable. Map, Filter, and Reduce can often help simplify your code, but aim for clarity over being concise. 5. Consider Alternatives: While these functions are often applicable, they are not always the most appropriate tool. Always consider the most suitable mechanism for your specific scenario.